hamhfrx-installer

Installation Guide — Phases 0 through 6

Station build automation

2026

1. What this installer does

Automates everything in the companion build manual up through a live, self-recovering, three-channel MP3 stream to Icecast:

1. **Configuration** — one interactive pass, asked once
2. **OS hardening** — SSH, firewall, fail2ban, unattended security upgrades, watchdog, SD-card-wear reduction
3. **Service account** — dedicated `hamsvc`, `/opt`-only file layout
4. **SDRplay API** — install and verify
5. **SDRangel** — built from source with the RSP2-capable plugin
6. **SDRangel configuration** — ALSA loopback (sized automatically to however many channels you configure), systemd units, demodulator channels provisioned via the REST API — any mix of SSB LSB, SSB USB, or AM, not a fixed set of three
7. **Streaming** — MP3 encode and push to Icecast, one systemd service per channel, indefinite automatic recovery from network/server outages

Not yet automated: Cloudflare Tunnel remote access, and the dormant SIP/baresip path. Both remain manual per the full build manual for now — see the notes at the end of this guide for why.

1.1 Design principles carried over from the manual build

- Everything lives under `/opt/`, nothing under a personal home directory
- A dedicated, unprivileged, no-login service account runs everything
- Every component is a systemd unit with correct startup ordering
- Every phase is **idempotent** — safe to re-run, and picks up where it left off rather than assuming a single flawless pass
- Configuration values used at runtime (ALSA device names, REST API behavior, sideband sign conventions) are the exact values verified against a real running system during the original build, not values assumed from documentation

2. Prerequisites

- Raspberry Pi 4 (8 GB recommended), Raspberry Pi OS Lite, **Bookworm (Debian 12), 64-bit** — not Trixie/Debian 13, which currently has an unresolved SDRangel source-build issue on aarch64
- SDRplay RSP2 Pro, connected via a short, good-quality cable to a USB3 port
- A regular user account with `sudo` rights (this is the account you SSH in as and run the installer as — **not** root, and **not** the `hamsvc` service account the installer creates for you)
- Internet access for package installation and the SDRangel source clone

3. Getting the installer onto the Pi

```
# however you prefer to transfer it — scp, git clone, etc.
unzip hamhfrx-installer.zip
cd hamhfrx-installer
chmod +x *.sh
```

> **On `curl | bash`:** this installer is deliberately not distributed for piping straight into a shell. Download it, look at it — at least skim the phase you're about to run — then execute it locally. Running an unreviewed script with root-capable `sudo` access is not a habit worth normalizing, regardless of the source.

4. Running it

Run every phase as your normal user — **never** with an external `sudo` prefix. Each script calls `sudo` internally only for the specific commands that need it, and will refuse to run (with a clear error) if launched as root directly.

```
./00-configure.sh
./01-hardening.sh
./02-service-account.sh
./03-sdrplay-api.sh
./04-sdrangel-build.sh
./05-sdrangel-config.sh
./06-streaming.sh
```

Every phase logs to `/var/log/hamhfrx-installer.log` in addition to your terminal, so you can review exactly what happened after the fact.

5. Phase-by-phase notes

5.0 — `00-configure.sh`

Interactive. Asks for hostname, admin username, SSH public key path, timezone, the three channel frequencies/modes, antenna port, sample rate, and (optionally, can be filled in later) Icecast and Cloudflare Tunnel details.

Writes `hamhfrx.conf`, `chmod 600`, since it holds the Icecast password in plain text. Every later phase reads from this file — nothing is asked twice. If you need to change a value later (add a fourth channel, rotate the Icecast password, etc.), edit this file directly and re-run the relevant phase.

5.1 — `01-hardening.sh`

Fully unattended. Sets the hostname, applies updates, switches SSH to key-only auth, enables `ufw` (default-deny inbound), configures `fail2ban` against the systemd journal (this OS ships without `rsyslog`, which trips up fail2ban's default log-file assumption), restricts unattended-upgrades to security-labeled packages only, sets timezone, disables Bluetooth, removes swap, and enables the hardware watchdog.

> If the SSH key step is skipped (no key path given, or the file wasn't found), the script pauses and asks you to confirm before disabling password authentication — this is the one place a mistake here would lock you out.

If the hostname changed, or the watchdog/`noatime` lines were just added to boot-time files, reboot before continuing:

```
sudo reboot
```

5.2 — `02-service-account.sh`

Fully unattended. Creates `hamsvc` (system account, no login shell), the `/opt/sdr-station-1` directory tree, and — the detail that cost real debugging time in the original build — adds `hamsvc` to **both** `plugdev` (USB/SDRplay access) and `audio` (ALSA access). Missing either one produces no obvious error; SDRangel just silently enumerates zero usable audio devices.

5.3 — `03-sdrplay-api.sh`

One manual step, by necessity. SDRplay's installer requires accepting a license interactively and has no stable long-term download URL (the filename includes a version number that changes), so this can't be fetched programmatically.

```
# get the current ARM64/aarch64 .run installer from
# https://www.sdrplay.com/downloads/ and save it into:
#     hamhfrx-installer/downloads/
./03-sdrplay-api.sh
```

Once the file is present, the rest is unattended: runs the installer (which will show its own license prompt), verifies the library and service, and checks the RSP2 Pro is visible on USB.

5.4 — `04-sdrangel-build.sh`

The long phase — 45–90+ minutes on a Pi 4. **Why source, not a prebuilt package:** community prebuilt arm64 Docker images for `sdrangelsrv` are commonly compiled without the SDRplay API present, which silently omits the RSP2-capable plugin (antenna port selection, external reference clock) and leaves only a legacy RSP1-only fallback. There is no runtime fix for a compile-time omission — building from source, with the API already installed, is the reliable path.

The actual `make` step runs fully **detached** (`setsid` / `nohup` / `disown`), so a dropped SSH session or tunnel disconnection does not kill the build. Watch progress with:

```
tail -f build/sdrangel-build.log
```

Re-running `./04-sdrangel-build.sh` at any point is the correct way to check on it:

- still compiling → reports that and exits
- just finished → runs `make install`, sets ownership, verifies the RSP2-capable plugin actually exists in the build output
- failed → says so clearly and points at the log rather than leaving you guessing

5.5 — `05-sdrangel-config.sh`

Fully unattended, but this is the phase with the most accumulated hard-won detail baked in:

- Loads the `snd-aloop` kernel module (three loopback card pairs) and makes it persistent — **and** adds an explicit systemd ordering dependency (`Requires=systemd-modules-load.service`) so `sdrangelsrv` can never start before the module is loaded. Without this, the loopback devices simply don't exist yet from SDRangel's point of view, which again produces no clear error — audio silently goes nowhere.
- Applies real-time CPU scheduling (`CPUSchedulingPolicy=rr`) to `sdrangelsrv`. Without it, all three channels' audio buffers overflow continuously even with CPU headroom to spare in `top` — this is a scheduling-latency problem, not a capacity problem.
- Computes each channel's frequency offset from the configured center frequency, and gets the LSB/USB sideband sign convention right (**both** `rfBandwidth` and `lowCutoff` go negative for LSB, positive for USB — this mirrors the passband around zero Hz, it is not two independently-chosen signs).
- Provisions the device set and three channels through the REST API in the specific order that actually works: create the channel bare, then `PATCH` its settings separately (this SDRangel version silently ignores settings embedded in the channel-creation call itself).
- Waits ~20 seconds after starting device acquisition for the RSP2 Pro's mandatory firmware upload to complete — this happens on **every** session open, not just the first one ever.

Ends by printing each channel's live signal report (`channelPowerDB`, `squelch`) so you can confirm real reception, not just a running process.

5.6 — `06-streaming.sh`

Requires `ICECAST_HOST` and `ICECAST_PASSWORD` to be set in `hamhfrx.conf` — if they were left blank during configuration, this phase tells you exactly what to edit and exits cleanly rather than half-running.

For each channel: writes a `chmod 600` credentials file (kept separate from the systemd unit, which would otherwise be world-readable), a capture-and-encode script, and a systemd service.

Two details worth knowing if you ever hand-edit these later:

- **`-ac 1 -channel_layout mono` together, not just `-ac 1`.** The first alone downmixes the input but doesn't guarantee the MP3 encoder itself emits a true single-channel stream — at 64 kbit/s the difference is audible, since true mono gives the full bitrate budget to one real channel of content.
- **The password-redaction `sed` filter.** ffmpeg prints its full connection URL — credentials included — into its own error messages. The generated script pipes ffmpeg's output through a filter that rewrites the password to `REDACTED` before it ever reaches the systemd journal.

`StartLimitIntervalSec=0` on every streaming service is the single most important line for genuine 24/7 operation. Systemd's normal default is to give up permanently after five restarts within ten seconds — fine for a bug, fatal for a real outage. Disabling that ceiling means ffmpeg retries every five seconds **indefinitely**, and reconnects on its own the moment the network or the Icecast server comes back, with no manual intervention required.

> To prove this rather than trust it: killing the network path with a firewall rule will **not** by itself interrupt an already-established TCP connection — that's normal stateful-firewall behavior, not a bug. To genuinely test recovery, sever the actual process:
>
> ```
> sudo pkill -9 ffmpeg
> sleep 6
> sudo systemctl status stream-ch1 --no-pager
> ```

> You should see a fresh restart and a clean reconnection within one `RestartSec` cycle.

6. Channel configuration — designed to be edited freely

Receiver channels are data, not code. There is no fixed channel count and no assumption that every channel is SSB — `channels.conf` is a plain, human-editable file that both the configuration wizard and the phase scripts treat as the single source of truth.

```
# freq_khz | mode | mountpoint | gain_db
3600  | LSB | ch1      | 10
7100  | LSB | ch2      |
4700  | USB | ch3      |
6000  | AM  | shortwave|
```

6.1 Why AM needed its own code path, not just another sign

SSB (LSB/USB) and AM are genuinely different at the SDRangel API level, not just a label difference:

- **SSB** uses a single filter with a **signed** bandwidth/low-cutoff pair — negative for LSB, positive for USB — because the passband sits entirely on one side of the carrier. This is the convention verified against the real running system during the original build (see the companion build manual, §6.2).
- **AM** has no sideband to select. SDRangel represents it with a different channel type (`AMDemod`) and a single **symmetric** bandwidth value, not a signed pair.

The installer branches on the `mode` column and generates the correct channel type and settings structure automatically — this was tested by generating a full provisioning script from a mixed LSB/USB/AM channel list and validating every embedded JSON payload independently before this design was included here.

> **One honesty note carried over from this whole build's approach:** the AM field names used (`AMDemodSettings` , `rfBandwidth`) match SDRangel v7.22.7 as built by this installer. If you're running a different version, verify with:
>
> ```
> curl -s http://127.0.0.1:8091/sdrangel/deviceset/0/channel/<n>/settings \
>   | python3 -m json.tool
> ```
>
> after the channel exists — the same "verify against the live system, don't trust documentation blindly" principle applied throughout this build, particularly relevant here since AM was not available to test against a physical receiver while this feature was written.

6.2 Mountpoint-based naming

Each channel's `mountpoint` value becomes both its Icecast mount **and** the name of its systemd service (`stream-<mountpoint>`). This is deliberate: it keeps a channel's identity stable even if you reorder `channels.conf` , rather than tying it to a position index that would shift under it.

6.3 Changing the channel list later

Edit `channels.conf` directly — add a line, remove one, change a frequency or mode — then re-run:

```
./05-sdrangel-config.sh
./06-streaming.sh
```

Phase 5 always restarts `sdrangelsrv` and reprovisions the device set from a clean slate when re-run. This is a deliberate simplification rather than a limitation: SDRangel's REST API has no notion of diffing an existing device set against a changed channel count, and a clean restart-and-reprovision is both simpler and more reliably correct than trying to patch live state. The ALSA loopback device count is resized to match automatically.

Phase 6 creates services for whatever's currently in the file, and **removes** the systemd unit, credentials file, and script for any mountpoint no longer present — shrinking the channel list cleans up after itself instead of leaving orphaned services silently running.

6.4 Center frequency and sample rate

The center frequency is computed automatically as the midpoint between your lowest and highest configured channel — not a fixed value. Both the configuration wizard and phase 5 warn (without blocking) if the spread between your outermost channels is tight relative to the configured sample rate, using the same ~80%-of-sample-rate usable-bandwidth rule of thumb established during the original build. If you see that warning, either widen `SAMPLE_RATE_HZ` in `hamhfrx.conf` for more margin, or accept reduced margin at the edges — both are legitimate choices depending on how much CPU headroom you're willing to spend.

7. Verification checklist after phase 6

```
sudo systemctl status sdrplay sdrangelsrv sdrangel-provision \
  stream-ch1 stream-ch2 stream-ch3 --no-pager

# USB stability — expect one clean firmware-load transition,
# not a repeating disconnect/reconnect cycle
sudo journalctl -k -b --no-pager | grep -i usb | tail -50

vcgencmd get_throttled     # expect 0x0

curl -s http://127.0.0.1:8091/sdrangel/deviceset/0/channel/0/report \
  | python3 -m json.tool

aplay -l | grep -i loopback

systemctl show stream-ch1 -p StartLimitIntervalUSec -p NRestarts
```

A full cold boot to fully-provisioned, streaming state takes approximately 2–3 minutes, dominated by the RSP2 Pro's mandatory firmware upload. This is expected, not a fault.

8. What’s intentionally left manual

Cloudflare Tunnel. The login step always needs one interactive browser authorization no matter how it’s scripted, and there is more than one reasonable way to structure remote access — a named tunnel with a Zero Trust Access application (email OTP), token-based tunnel creation for a more scriptable first run, WARP-based private network access instead of a public hostname, or a different identity provider behind Access. Rather than hardcode the one approach used in the original build, this is left for a future phase that asks which model fits your situation, since it’s the piece most likely to vary by where and how the station is actually managed. Follow the build manual’s Cloudflare Tunnel section in the meantime.

SIP / baresip. Fully designed in the build manual (outbound-only registration, per-instance ports, firewall rules scoped to the known Asterisk IP) but intentionally left dormant and unscripted, since it depends entirely on credentials and server details that don’t exist until you have a specific Asterisk deployment to point at.

Antenna and physical RF decisions. Antenna port and reference clock are configuration values this installer will happily apply, but *which* values are correct depends on your actual antenna and site — those remain a human decision, not something a script should guess.